

Supplementary material

Authoritative-State Admission for AI-Derived Updates
Artifact details, revision checks, and checklist example

The companion main manuscript retains the admission contract, Proposition 1, primary evaluation methods and results, and validity limits. Section and table numbers beginning with S refer to this supplement.

S1. Artifact and evidence boundary

The paper draws on separately manifested evidence layers: the formal/concrete and fixed-grid baseline, an AI-origin lifecycle fixture, a pinned agent-harness integration, performance extensions, and the repeated live-agent Final. The repository retains immutable source, configuration, dependency locks, raw receipts, normalized inputs, generated tables, and claim–evidence lineage for each layer. Separating these layers prevents later experiments from silently changing the baseline denominators.

Baseline quantitative and pass/fail statements are generated from exactly the ten normalized inputs in table S1. The AI-origin and harness studies add `ai_origin_lifecycle_summary.json` and `dsh_plugin_integration_summary.json`, respectively. The performance extensions add `r21_feature_baseline_summary.json` and `r21_trace_optimization_summary.json`. The repeated benchmark enters through the no-overwrite staging bundle `2026-09-03-three-config-10x`, whose status is `READY_FOR_MANUSCRIPT_INTEGRATION`. It contains all 1,260 normalized Final rows, aggregate statistical analysis, two generated tables, the endpoint figure, and its own manifest. Each layer retains raw-source references and claim lineage.

Table S1: Exact ten-input baseline set. Roles and per-file lineage are recorded in the artifact repository.

Inputs 1–5	Inputs 6–10
<code>run_summary.json</code>	<code>lifecycle_summary.json</code>
<code>quality_summary.json</code>	<code>cost_summary.json</code>
<code>alloy_results.json</code>	<code>cost_run_metadata.json</code>
<code>formal_refinement_summary.json</code>	<code>correctness_environment.json</code>
<code>conformance_summary.json</code>	<code>performance_environment.json</code>

The table generator checks the exact input set, recomputes every input hash, and verifies that correctness and performance metadata bind the declared source, configuration, and dependency identities. The extension manifests bind performance modules and configurations. The repeated- benchmark Final binds the runner, model/matrix configuration, tool surface, raw attempts, host actions, normalized rows, generated tables/figure, and direct source dependencies. Its verified parent-manifest SHA-256 is `6e41df602fab195bedbfa5ec97d152b5921b0ef0cace6f8da7c5204b6cd433ec`. Whole-package manifests are non-self-referential and detect missing, extra, or modified files.

Per-file SHA-256 values, receipt paths, clean-extraction checks, and one-byte tamper-probe outputs are maintained in the artifact repository rather than repeated in the manuscript. Diagnostic and failed runs remain available as provenance but are excluded from normalized paper inputs. The repository README, Final report, and manuscript-input status identify the citable runs and provide the regeneration and verification commands.

The v8 observation witnesses and frozen R21 experimental evidence are deposited at <https://github.com/lhh666-6/auto-dete/tree/r21-jss-2026-09-07-v8/r21-jss>. Its package-level and revision-level manifests identify the v8 manuscript and evidence. The accompanying submission candidate places the current manuscript, repaired code, witnesses, and derived analyses under its current manifest and groups this revision with an unchanged v8 export under a release-level manifest. The cited v8 URL is historical; the current candidate has not yet been published. Figures 1 and 2 present the admission workflow and contribution map.

The r22 review-response revision adds four versioned evidence items without replacing the frozen v8 records: the canonical value-equality repair with thirteen regression tests (`evidence/r1-value-equality/`), the minimal-repair rule re-score of the frozen acknowledgment audit (`evidence/r3-audit-verified/`), the strict-versus-endpoint task sensitivity analysis over the frozen events (`evidence/r4-endpoint-verified/`), and the corrected observation-witness report with the copy-forward control (`evidence/r5-witness/`). The baseline copy of the v8 manuscript and its PDF are retained byte-identically in the revision directory. The frozen performance measurements describe the v8 implementation and were not rerun for the value-equality repair.

The r27 revision adds `evidence/r27-trace-mutation/`: a positive bounded assertion that legal Full admission preserves trace completeness under an explicit well-formed-pre-state condition, together with a mutation that

Table S2: Paired admission measurements on the frozen configuration (200 measured pairs per cell). Negative deltas are retained.

Fields	Changed	Lower p50	Full p50	Full p95	Mean delta milliseconds	95% CI
1	1	8.964	16.573	18.435	7.807	[7.451, 8.218]
8	1	10.048	17.176	19.756	7.542	[7.126, 8.058]
8	4	9.841	20.997	22.807	11.405	[10.977, 11.902]
8	8	10.030	26.459	28.536	16.407	[16.064, 16.742]
32	1	13.541	17.285	19.242	3.831	[3.642, 4.036]
32	4	13.611	22.060	41.323	10.064	[9.372, 10.909]
32	32	13.673	58.467	61.749	45.379	[44.874, 45.946]
128	1	24.580	18.590	22.465	-5.799	[-6.536, -4.999]
128	4	24.756	23.126	26.624	-1.972	[-4.129, -0.598]
128	128	25.490	185.532	199.407	160.681	[159.699, 161.633]

deletes the post-state certificate binding and flips that assertion from UNSAT to SAT. The assertion makes the pre-state dependency explicit instead of assuming it; the concrete service implements the same dependencies through predecessor-prefix trace validation, the per-record uniqueness constraint on fact transitions, and compare-and-swap admission with in-transaction revalidation. The bounded catalogue now contains 34 commands per profile (68 total).

S1.1. Historical cost details

The main text reports the frozen v8 persistence-equivalent and optimized measurements. For complete accounting, table S2 retains the historical lower-bound admission comparison and table S3 the historical reverse-trace summary. These tables describe the frozen v8 measurements and are not used to isolate the cost of any single check.

S1.2. Historical zero-event calculation not used for inference

The historical analysis file retains an arithmetic zero-event calculation over 899 operations. This revision does not interpret it as a population bound: the workload mixes fixed admission probes and capability-unavailable branches and does not establish independent Bernoulli sampling. The manuscript reports operation counts separately.

Table S3: Reverse-trace latency summarized over the three record populations. Each underlying cell has 200 trials.

Fields	Versions	p50 range (ms)	Maximum p95 (ms)
1	1	5.327–5.358	5.968
1	10	11.799–12.438	12.924
1	100	77.427–79.522	82.076
8	1	17.348–17.900	18.770
8	10	44.601–46.203	47.977
8	100	319.761–324.638	346.507
32	1	58.404–59.986	62.551
32	10	156.024–161.123	164.620
32	100	1127.005–1175.999	1199.629
128	1	222.453–253.617	284.498
128	10	651.482–672.763	689.231
128	100	4794.968–7857.313	8046.959

S2. Revision checks and historical protocol

S2.1. Value equality and witness domains

A cross-layer review of the previous build found that the planner and trace validator decided whether a field changed with Python equality, while the independent oracle and formal projection used canonical JSON equality. Because Python identifies 2 with 2.0 and 1 with `true`, a submission through the public confirmation API could commit a new serialized value while reusing the predecessor source, after which the trace immediately reported `incomplete`. The repair routes every changed-field decision through one canonical-equality relation. Thirteen regression tests cover an unchanged equal-value control, integer-to-float and integer-to-boolean changes, a nested-value change, a single-field change, and a fail-closed pure no-op; seven fail against the previous implementation and all pass against the repaired one, and the affected conformance workloads were rerun with the complete suite green. The observation-witness checker was corrected in the same revision to separate the schema field domain from the declared item field domain: an ordinary single-field update with another field copied forward is now a complete successor rather than a partial one, while the five original pairs retain their derived outcomes. The checker validates the five declared constructions

plus the candidate-identity pair and one copy-forward control; it is not a general admission oracle.

The current checker additionally isolates candidate identity in a separate D_C pair while retaining all five original pairs and the copy-forward control. Its versioned report is `evidence/r27-witness/observation-witness-report-v5.json`; the earlier report remains historical. Nineteen focused witness tests pass, including three new checks of identity isolation and report coverage.

S2.2. Textual rule audit

The original recognition fields used lexical indicators (`mismatch/differentrecord` and `stale`), which can miss synonyms or count negated and identifier-only occurrences. A post-hoc rule-based audit rescored all behavior-evaluable A2/A3/B4/A6 outputs: 174 context and 151 stale outputs. One author specified the rubric; a deterministic script assigned every label from assistant text with model configuration and old labels withheld. No AI model or independent human annotator assigned row labels, so these are *rule-hit counts*, not validated semantic measurements. This statement concerns the A2/A3/B4/A6 textual indicators; the B1–B4 benign completion endpoint was separately human-annotated by one author (X.W.) and one non-author volunteer (Supplement S2.7). A review of the previous build found two rule defects: an explicit denial (“no mismatch”, “no cross-field conflict arises”) could be counted as a positive hit, and synonymous assertions (“matches X, not Y” versus “X does not match Y”) could receive opposite labels. A versioned minimal-repair re-score (v2.1) addresses those two defect classes, including denial scope and case-normalized field identifiers and otherwise retains the previous rule outcome; it changes 11 of 325 rows and leaves the stale count unchanged. The repair uses the historical rule labels as input; it is not an independent blinded annotation. We report the historical and repaired counts and retain every disagreement in the artifact.

The A2 and A3 context scenarios are reported separately. A2 supplies a cross-record challenge whose intended record is explicit. A3’s frozen prompt did not name a unique intended target field: all 90 prepared A3 inputs omit an explicit intended/target key, and the challenge certificate equals the parent of the second listed field while the host separately targets quantity. An A3 response that matches the listed field is therefore a defensible reading of the task, and A3 is reported as an exploratory response count rather than a context-mismatch acknowledgment rate. The A2 and A3 counts are not merged. These textual indicators record rule matches related to context

and freshness; they do not establish semantic acknowledgment or general recognition ability. Complete row coverage by a single-rubric post-hoc script does not establish measurement validity. Unavailable-capability attempt remains A10, and recovery is covered by B4.

S2.3. Historical admission comparator

For the historical lower-bound comparison, the full-arm latency ends when the review service’s complete authority transaction commits; a successful post-admission trace is then checked outside the timed region. Its paired lower-bound comparator performs only a minimal version CAS, record-snapshot insert with an empty source map, and field-value updates. The arms run in randomized order on fresh clones of one initialized database per pair. Unique combinations of 1, 8, 32, and 128 fields with 1, 4, or all fields changed yield ten cells and 2,000 pairs. We report full p50/p95 and the paired mean latency difference with a 2,000-draw nonparametric 95% bootstrap interval. Differences are interpreted within this lower-bound comparator design.

S2.4. Lifecycle and pinned-plugin controls

These controls distinguish archived-input compatibility from runtime-interface conformance. They complement the main manuscript’s lifecycle and live-agent results.

AI-origin lifecycle probe. One archived hosted-AI fixture retains the prompt, response, normalized candidate, generation metadata, and a five-file manifest; backend revision and decoding settings are unavailable. Confidence is uncalibrated and excluded from admission. Six fresh-database cases run once: all-field Accept, singleton/all-field/mixed Correction, stale replay, and cross-field substitution. The first four require expected versions and complete traces; the latter two require rejection and an unchanged independent database digest.

DeepSeek Harness plugin probe. The pinned plugin and dependency lock are driven keylessly through Cordis and DSH `ToolRuntime`. Ten cases cover tool composition, proposal, host Accept/Correction, stale and evidence rejection, absent confirmation/bridge, unloading, and manifest verification. Four rejection cases require unchanged authority digests; a one-byte receipt mutation must be localized exactly.

All six AI-origin cases matched their predeclared outcomes: the four Accept/Correction cases produced the expected versions and traces, while stale

replay and invalid-item substitution rejected without changing the canonical database digest. The raw records and `ai_origin_lifecycle_summary.json` retain the case details. Together, the two lifecycles demonstrate candidate preservation, authorized correction, copy-forward, and fail-closed admission for the declared synthetic/public inputs.

All ten predeclared DSH cases passed. Only proposal and verification were model-callable; proposal left the fact version at zero. Host Accept created one successor, while Correction retained the proposed value 8 and authorized 9. All four rejected-action cases preserved the authority-state digest. Unloading removed the tool schemas but retained readable records. The clean 30-file manifest verified, and the one-byte tamper probe identified exactly `receipt.json`.

These results establish a deterministic runtime-integration and external-checkability example for the pinned release. RQ8 separately exercises live model behavior on the restricted surface.

S2.5. Export reproducibility and observation scope

The historical normalized quality summary records 354 tests for its own snapshot; later v8 source collection and revised development-suite counts are different versioned inventories, not replacements for that summary. The prior 377-test result depended on ancestor-repository Alloy resources and Git metadata. The current export includes the Alloy model and portable Windows Java/Alloy runtime, and benchmark metadata records unavailable Git state as unknown rather than failing or claiming a clean checkout. Its isolated verification report records the actual test inventory and environment; no tests are skipped to conceal missing dependencies.

Proposition 1 concerns five declared observation classes with principal membership fixed. It does not establish that these exhaust authentication or dynamic-policy failures. The following inspection matrix separates abstract histories from operational probes. A rejected public call produces a stutter, not the unsafe persisted history itself.

Two controlled revocation schedules exercise the stated policy boundary: a second thread completes revocation after the successful policy check, at the before-CAS or before-commit failpoint. The in-flight admission remains permitted under commit-entry semantics. These checks do not claim linearizable revocation; the existing pre-check revocation control requires rejection. No sixth observation class or stronger policy contract is inferred.

Table S4: Interpretation of constructed histories and concrete probes.

Class	Ordinary-interface probe	Unsafe-history interpretation
D_C	Submit a substituted candidate; binding checks reject	A committed mismatch describes a missing/bypassed check, not an accepted Full execution.
D_V	Legal Correction preserves proposal and authorization	Erased or relabeled roles are observation loss or corrupted representation; legal Correction is not itself unsafe.
D_F	Replay after a newer version; CAS rejects	A stale commit requires loss/bypass of the stated freshness enforcement.
D_B	Inject an intermediate failure; transaction rolls back	Fragmented commits describe a weakened transaction path, not partial success of the full one.
D_S	Check exact source resolution and copy-forward	Corrupt-source fixtures use isolated storage; schema prevention and post-storage detection are distinct outcomes.

S2.6. Retrospective scenario and endpoint audit

The input audit checks all 90 inputs per scenario: each prompt embeds its corresponding prepared context, field keys are unique, and declared proposal fields belong to the listed field set. The following source-level interpretation also reads the frozen prompt tasks, scoring rules, and host branches. Passing these mechanical checks does not establish natural-language construct validity. A5, like A3, does not supply a uniquely designated intended-evidence key; no A5 semantic-recognition rate is inferred. A6/A8 list stale field metadata, so freshness must be resolved through the tool against current state rather than inferred from that list alone.

The less restrictive endpoint criterion removes sequence/lexical restrictions while checking the declared terminal events. Its higher observed count is not an independent validation of the frozen strict measure. The manuscript retains the comparison as post-hoc sensitivity and removes it from the abstract. No score or frozen model response is overwritten by this audit.

S2.7. Human annotation of benign completion (B1–B4)

One author (X.W.) and one non-author volunteer independently coded all 360 planned benign runs under a frozen completion rubric: 1 = the reported tool results reached the task’s terminal state (the declared value was proposed

and its returned certificate was verified), 0 = the terminal state was not reached or the response over-claimed host authority, and 9 = the response was truncated, lacked tool results, or could not otherwise be determined. Both used the same coding manual and the same English task/response text. The non-author annotator was blind to the deterministic rule labels, model configuration identifiers, prompt variants, repetitions, and study hypotheses; the author annotator was blind to the rule labels and configuration identifiers at coding time but, as an author, was not blind to the study hypotheses. An optional LLM-assisted Chinese translation was available as a non-authoritative reading aid, with use logged. The frozen scorer’s 25 non-evaluable runs were not folded into the primary 335-run comparison.

On the full 360-run set, inter-human agreement was 359/360 (99.72%; Cohen’s $\kappa = 0.974$, 95% CI [0.911, 1.000]; AC1 = 0.997; PABAK = 0.996). The only disagreement (T153) was adjudicated to 0 because the response verified the stale certificate but did not report the replacement proposal or the final verification. Against the strict rule on the 335 behavior-evaluable runs, each annotator agreed on 324/335 (96.72%; AC1 = 0.965; PABAK = 0.934; $\kappa = 0.410$, 95% CI [0.113, 0.661]) before adjudication; all 11 disagreements were cases in which both humans judged completion while the strict rule did not. After author adjudication of these 11 cases and T153, the adjudicated labels matched the strict rule on all 335 behavior-evaluable runs (320/335 completions). Of the 25 unscored runs, both annotators labeled 15 as 9, both labeled 8 as 1, both labeled one as 0, and one (T153) was adjudicated to 0. Because one coder was an author and the adjudicator was the other author, this is author-involved annotation and author adjudication rather than independent third-party annotation; the 11 pre-adjudication human–rule disagreements remain in the disagreement lists and are not overwritten. The analysis archive retains confusion matrices, prevalence, AC1/PABAK, case-, model-, and cell-level bootstrap intervals, and the adjudication file.

Table S6 reports the case-, model-, and cell-level bootstrap intervals. The human–rule κ interval widens from [0.113, 0.661] at the case level to [0.000, 0.885] under model-level resampling and [0.000, 0.831] under cell-level resampling; the inter-human interval is already wide at the case level and does not narrow the conclusion. Table S7 reports the benign strict-trajectory completion count and the missingness extremes for each configuration. G1 has 1 unscored run, G2 and D1 each have 12; their planned-run bounds overlap, so the aggregate endpoint does not establish a model ranking.

Table S5: Scope of the fourteen frozen scenarios; host challenge tuples are fixed independently of the model answer.

Scenario	Visible task anchor	Supported endpoint and limitation
B1	One declared field/value	Proposal/verification completion; strict sequence may reject extra calls.
B2	Proposal and authorized value separated	Model proposes/verifies; the host performs Correction, not the model.
B3	Three declared field/value pairs	Multi-field proposal/verification, not model-controlled atomic admission.
B4	Designated stale and fresh certificates	Ordered recovery endpoint; strict also requires the word stale.
A1	Declared proposal plus untrusted confirmation text	Tool-boundary behavior; unavailable host branch makes no admission call.
A2	One intended record and challenge certificate	Cross-record text indicator plus separate host rejection; no validated semantic rate.
A3	Two fields, no unique intended target	Exploratory text only; host-selected target cannot supply a model-visible answer key.
A4	Authorized and attempted candidate identifiers	Host candidate-binding probe; verification alone does not measure full identity reasoning.
A5	Challenge and foreign-evidence identifiers	Host evidence-substitution probe; intended-evidence semantics not independently validated.
A6	Designated stale certificate	Stale text rule hits and separate host replay rejection; current state comes from verification.
A7	Proposal/authorized/attempted values explicit	Host authorized-value check; model output is not the invalid host tuple.
A8	Previously committed challenge certificate	Host replay check; not a standalone semantic-recognition measure.
A9	Full declared proposal set, partial host challenge	Host batch-integrity probe; no model-generated partial-admission attack.
A10	Confirmation requested only if exposed	Capability absence and attempted-use behavior; no admission call or security population estimate.

Table S6: Human-annotation agreement under case-, model-, and cell-level bootstrap resampling. The reported main-text intervals are case-level; the model- and cell-level intervals show the wider uncertainty under clustering. AC1 and PABAK are agreement coefficients that do not depend on the bootstrap unit.

Comparison	N	κ	Case-level 95% CI	Model- level 95% CI	Cell-level 95% CI	AC1	PABAK
A1–A2 (1/0/9)	360	0.974	[0.911, 1.000]	[0.952, 1.000]	[0.937, 1.000]	0.997	0.996
A1–rule (1/0)	335	0.410	[0.113, 0.661]	[0.000, 0.885]	[0.000, 0.831]	0.965	0.934
A2–rule (1/0)	335	0.410	[0.113, 0.661]	[0.000, 0.885]	[0.000, 0.831]	0.965	0.934

Note: A1–A2 is inter-human agreement; A1–rule and A2–rule are human–rule agreement, not inter-human agreement. The A1–A2 1/0/9 row uses the full 360-run set; the human–rule rows use the 335 behavior-evaluable runs.

Table S7: Benign B1–B4 strict-trajectory completion by model configuration. The planned-run bound treats the unscored runs as all failures or all successes; it is not a confidence interval and does not establish a model ranking.

Configuration	Planned	Evaluable	Unscored	Strict complete	Strict/ Planned bound	Strict/ Planned bound
G1 (gpt-5.6-luna)	120	119	1	114	95.80%	95.00– 95.83%
G2 (gpt-5.6-terra)	120	108	12	105	97.22%	87.50– 97.50%
D1 (deepseek-v4-flash)	120	108	12	101	93.52%	84.17– 94.17%
All configurations	360	335	25	320	95.52%	88.89– 95.83%

S3. Executable design comparison

The original executable comparison declares two SQLite policies over nine fixed cases, and this section adds a third minimal ordinary approval/audit-log baseline. The standard-library Python demonstrator makes no hosted-model calls, leaves prior frozen experiments unchanged, and measures no performance.

Both policies preserve immutable candidate rows, complete version snapshots, principal/value/domain-bound approvals, atomic compare-and-swap (CAS), and complete before/after value audits. The rejection and rollback controls in table S8 compare every application table before and after submission, with approvals already persisted. The *value-audit* policy stores the approved values and the expected version but omits which persisted candidate was inspected. The *candidate-bound* policy additionally persists the reviewed candidate identity, rechecks it on submission, and stores one exact source reference per authoritative field, copying unchanged-field sources forward.

Table S8 reports the nine fixed cases. The two policies agree on the accept/reject outcome in seven of the eight single-run cases and diverge in one: under equal-valued candidate substitution, the value-audit policy commits the approved value because the substituted candidate has the same record, field, and value, whereas the candidate-bound policy rejects the submission with `candidate_binding`. The paired histories are the sharper observation. Two runs review candidate 100 and candidate 101 and both approve and commit 101 with identical candidate pools, values, actor, and expected version. The complete logical application-table state is identical across the two value-audit runs and differs across the two candidate-bound runs; the difference is exactly the persisted reviewed-candidate identity. This shows omitted review linkage in the conventional policy, not the absence of candidate values, which both policies retain. The candidate-bound policy also reconstructs the proposed value by joining the persisted candidate rather than duplicating it, so the proposed value 100 and the authorized value 101 remain separately attributable. Both policies reconstruct the same per-field sources from the audit history in the two-step case; only the candidate-bound policy persists the source map. This control demonstrates no source-reconstruction advantage for the added map: the complete baseline audit already supports that observation. The demonstrated increment is exact reviewed-candidate binding and the resulting proposal/authorized-value attribution, not a requirement that D_S use an explicitly stored pointer.

Table S8: Executable comparison of two declared SQLite policies over nine cases fixed before execution. Both share immutable candidates, complete version snapshots, value/domain/version-bound approvals, atomic CAS, and complete before/after audits. † marks the only accept/reject divergence; ‡ the paired-history observation. The demonstrator is illustrative, not a production implementation or a statistical comparison.

Case (fixed before execution)	Requested action	Value-audit policy	Candidate-bound policy
Legal correction	approve q100a; submit q100a (101)	accept; v1 quantity=101	accept; v1 quantity=101
Equal-valued substitution†	approve q100a; submit q100b (101)	accept; committed 101	reject candidate_binding ; unchanged
Cross-record candidate	approve q100a; submit other100	reject candidate_context ; unchanged	reject candidate_context ; unchanged
Stale expected version	submit review-1 after intervening commit	reject stale_version ; unchanged	reject stale_version ; unchanged
Wrong submitted value	approve 101; submit 102	reject approved_values ; unchanged	reject approved_values ; unchanged
Partial approved batch	approve quantity+batch; submit quantity	reject approved_values ; unchanged	reject approved_values ; unchanged
Injected failure after writes	submit with failure after writes	reject injected_failure ; unchanged	reject injected_failure ; unchanged
Two-step copy-forward	commit quantity 101; then batch B	accept; sources batch=v2:batch, quantity=v1:quantity	accept; sources batch=v2:batch, quantity=v1:quantity; stored
Paired reviewed-candidate histories‡	review q100a vs q101; both commit 101	logical table state identical	logical table state differs in approvals

S3.1. Minimal ordinary approval/audit-log baseline

A third, deliberately conventional SQLite policy was executed on the same nine cases as an external standard-practice comparator. It stores approved values, expected version, principal, immutable candidates, and a complete before/after audit log, but no reviewed-candidate binding and no explicit source map. It validates candidate record/field context, approved-value equality, changed-field domain, and version compare-and-swap, and it can reconstruct every field’s source by diffing the complete audit history. It agrees with the candidate-bound policy on the accept/reject outcome in seven of the eight single-history cases and diverges only on equal-valued candidate substitution: approving candidate q100a and submitting the equal-valued q100b is accepted by the ordinary policy and rejected by the candidate-bound policy. In the paired reviewed-candidate histories, the ordinary policy’s persisted state is identical for q100a and q101, whereas the candidate-

bound state differs in the approval binding. Its proposal/authorized-value attribution is also ambiguous because the approval row stores no candidate identity and the registry contains equal-valued candidates for the same field. The baseline therefore isolates the demonstrated increment to exact reviewed-candidate binding and unique proposal/authorized-value attribution; it does not compare against event-sourcing/CQRS, temporal/bittemporal tables, or four-eyes approval systems. Its runner, nine per-case SQLite databases, and summary are retained under `evidence/r27-standard-practice-baseline/`.

The demonstrator is an illustrative specialization, not a second production implementation of the Auto-Decte contract. It does not implement cryptographic evidence verification, enterprise authentication, distributed revocation, race stress, or performance measurement, and it is not evidence that any deployed system is defective. The nine cases are a deterministic construction; the counts are not a statistical or superiority claim. The runner, raw SQLite observations, and derived table are retained in the package.